OC

iscte INSTITUTO UNIVERSITÁRIO DE LISBOA

Optimization for 3D Scene Reconstruction

A Comparative Study of Optimizers, Losses, and Representations for NeRF and 3D Gaussian Splatting

António Cruz (140129), **Duarte Cabrita** (120058)

Table of Contents

1. Introduction

This project studies computational optimization through an applied problem in computer vision: reconstructing a 3D scene from a small set of 2D photographs so that new, previously unseen viewpoints can be synthesized.

The task underpins applications across robotics, augmented and virtual reality, autonomous driving, virtual production, and digital heritage preservation. Two state-of-the-art representations frame the reconstruction problem as continuous, non-convex optimization:

- **Neural Radiance Fields (NeRF)**: the scene is a small multi-layer perceptron mapping a 3D point and a viewing direction to a color and density, fitted by gradient descent through a differentiable volume renderer.
- **3D Gaussian Splatting (GS)**: the scene is an explicit collection of 3D Gaussian primitives, each with a position, scale, rotation, opacity, and color, fitted by gradient descent through a differentiable rasterizer.

The project is deliberately and exclusively an **optimization study**. It does not propose a new scene representation, nor extend NeRF or GS as published; what the project contributes is a controlled investigation of how the optimization choices around these two representations shape convergence behavior, training stability, and final reconstruction quality.

The optimization layers compared in the sections below are:

Table 1 - Optimization layers built in this project and the sections in which each is presented.

Layer	Component	Where it is discussed
Inner gradient loop (model parameters)	Five first-order optimizers implemented from scratch (SGD, momentum, Nesterov, Adam, AdamW), plus a cosine-annealing schedule with linear warmup	Section 4 (implementation), Section 7 (comparison)
Loss formulation	Four candidate losses (L2, L1, SSIM, L1+SSIM) compared under the same training procedure	Section 5 (implementation), Section 8 (comparison)
Outer hyperparameter loop	Bayesian optimization of L1's learning rate with the Tree-structured Parzen Estimator, early-stopped by a median pruner	Section 8.2
Substantiated improvements	Four candidate improvements (SGDR learning-rate restarts, perceptual loss, multi-scale coarse-to-fine training, adaptive view sampling), each motivated by a specific property of the optimization problem	Section 9
Alternative parameterization	A 3D Gaussian Splatting implementation built on a published reference rasterizer, with four non-obvious implementation details required to recover correct training (Section 10.2)	Sections 10 and 11

2. Problem Formulation

This project implements 3D scene reconstruction from a sparse set of 2D photographs, framed as a continuous non-convex optimization problem.

2.1 Mathematical formulation

Let θ be the parameters (weights and biases) of a small multi-layer perceptron f_θ that maps a 3D point (x, y, z) to a density $\sigma \geq 0$ and an RGB colour $c \in [0,1]^3$.

Let $R(\theta; \pi)$ be the differentiable volume-rendering operator. Given θ and a camera pose π, it casts rays through every pixel, samples points along each ray, queries f_θ, and composites the samples into a synthesised image.

Given captured images and their camera poses $\{(I_i, \pi_i)\}_{i=1..N}$, the reconstruction minimises the average discrepancy between rendered and observed pixels:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(R(\theta; \pi_i),\, I_i\right)$$

where $\mathcal{L}$ is one of the loss formulations (the baseline being the squared error $\|\cdot\|_2^2$).

The optimum is characterized by the **first-order optimality condition** $\nabla_\theta \mathcal{L} = \mathbf{0}$, the same condition introduced in Module 1's classical theory of unconstrained optimization.

However, two features of this problem make Module 1's analytical machinery inapplicable: the loss is **non-convex** (the rendering operator composes with the MLP non-linearities) and **high-dimensional** ($|\theta|$ on the order of 10^5 parameters even for the small model used here).

The non-convexity rules out a closed-form solution of $\nabla_\theta \mathcal{L} = \mathbf{0}$; the dimensionality rules out the Hessian-based classification that distinguishes minima from saddle points analytically (the Hessian alone would have on the order of 3×10^9 entries for this model).

The problem is therefore solved by **iterative first-order stochastic gradient methods**: at each iteration a random image and a random batch of its pixel rays are drawn, rendered, scored against the ground truth, and used to update θ along $-\nabla_\theta \mathcal{L}$. The choice of *which* gradient-based method to use is itself the central question this project studies.

The formulation above leaves three components open, and the project varies each in a controlled way:

- **The optimizer** that performs the update $\theta \leftarrow \theta - \cdots$.
- **The loss** $\mathcal{L}$ that defines the objective surface.
- **The scene representation** itself, NeRF versus 3D Gaussian Splatting.

Convergence speed, training stability, and final reconstruction quality are measured for each, under identical conditions, using the methodology of Section 6.

3. NeRF: Model and Differentiable Renderer

The synthetic experiments use the nerf_synthetic dataset of Mildenhall et al. 2020, eight scenes rendered in Blender with known camera poses. The self-captured scene of Section 12 is parsed from COLMAP's sparse reconstruction output and converted into the same image / pose / focal-length representation, so the same training and evaluation code applies to both.

The NeRF model follows Mildenhall et al. 2020. A small fully connected MLP maps a 3D point, lifted into a higher-dimensional feature space by a fixed sinusoidal positional encoding, to a non-negative density and an RGB colour. The architecture is four layers of width 128, about 58,000 parameters, selected by the scoping study of Section 6.1: a substantially larger network raised reconstruction quality by only a few tenths of a dB for several times the per-iteration cost, which is not a worthwhile trade in a study whose subject is the optimizer rather than absolute reconstruction quality.

The differentiable rendering operator $R(\theta; \pi)$ of Section 2 is realised by generating one ray per pixel, sampling N points along each ray between the near and far planes, querying the MLP at each sample, and compositing the per-point predictions into a per-pixel colour through the discretised volume-rendering integral with alpha-compositing weights derived from accumulated transmittance. The operator is differentiable end to end, so gradients of the image loss flow back into the MLP weights through autograd. Full-image rendering for evaluation uses the same integral in ray chunks to bound memory consumption.

4. Optimizers

This project implements five gradient-based optimizers from scratch, on top of PyTorch automatic differentiation. Implementing them ourselves rather than calling a library is what makes the comparison a genuine study of computational optimization, and it guarantees every method runs under identical numerical conventions: same float precision, same in-place arithmetic, same boundaries between trainable and non-trainable state. Any difference observed in Section 7 is therefore attributable to the algorithm itself, not to engineering differences between library implementations.

All five methods are **first-order** (gradient-only). They iteratively approach the first-order condition $\nabla_\theta \mathcal{L} = \mathbf{0}$ introduced in Section 2, but none performs the second-order analysis (Hessian eigenvalues, principal-minor / Sylvester tests) that Module 1's classical theory uses to classify the resulting critical point: forming the full Hessian for $|\theta| \approx 58{,}000$ would require on the order of 13 GB just to store, and using it inside an optimizer step is intractable at this scale.

Adam's second moment estimate $\hat{v}$ acts instead as a *diagonal* approximation of curvature, a tractable surrogate for the second-order information the analytical theory uses directly. This is the trade-off the field makes for problems of this size, and the comparison in Section 7 quantifies what is lost and gained by each of these substitutions.

A cosine-annealing learning-rate schedule with linear warmup can be applied on top of any of the five methods; its effect is one of the comparisons in Section 7.

4.1 Stochastic Gradient Descent

Plain SGD is the reference baseline: each parameter moves directly down its gradient, $\theta \leftarrow \theta - \eta\, g$, with a single global step size η. It has no state and no per-parameter scaling, so it is the most exposed to ill-conditioning, directions of high curvature force a small η, which then makes progress along low-curvature directions slow.

4.2 SGD with Classical Momentum

Momentum accumulates an exponentially weighted running sum of past gradients, the velocity $v \leftarrow \mu v + g$, and steps along it: $\theta \leftarrow \theta - \eta\, v$. The decay μ (here 0.9) damps the oscillation that plain SGD suffers across high-curvature directions, while letting consistent descent directions build up speed. This usually gives faster and steadier convergence.

4.3 SGD with Nesterov Accelerated Gradient

Nesterov's accelerated gradient evaluates the descent direction with a look-ahead: the velocity is updated as in classical momentum, but the step applied is $\theta \leftarrow \theta - \eta\,(g + \mu v)$, which corresponds to measuring the gradient slightly ahead, where the momentum term is about to carry the parameters. The correction reduces overshoot near the minimum and, on well-behaved objectives, improves the convergence rate.

4.4 Adam

Adam keeps two exponentially weighted moment estimates per parameter: the first moment m (a smoothed gradient, like momentum) and the second moment v (a smoothed squared gradient). The update divides the first moment by the square root of the second, $\theta \leftarrow \theta - \eta\, \hat{m}/(\sqrt{\hat{v}} + \epsilon)$, giving every parameter its own effective step size, large where gradients are small and consistent, small where they are large or noisy. Both estimates start at zero and are therefore bias-corrected ($\hat{m}, \hat{v}$) so the early steps are not damped. This per-parameter adaptation is what lets Adam cope with the ill-conditioning that slows plain SGD.

4.5 AdamW

AdamW is Adam with *decoupled* weight decay. Standard L2 regularization adds $\lambda\theta$ to the gradient, so it passes through Adam's per-parameter scaling and is effectively rescaled differently for every weight. AdamW instead applies the decay straight to the parameter, $\theta \leftarrow \theta - \eta\lambda\,\theta$, separately from the adaptive gradient step. The regularisation strength is then uniform and independent of the gradient statistics, which is the behaviour usually intended by "weight decay".

4.6 Learning-Rate Schedule

Each of the five optimizers takes a fixed base learning rate. A schedule modulates that rate over training, independently of which optimizer is used. The schedule combines a short *linear warmup*, the rate ramps up from zero over the first few hundred steps, avoiding a destructive early step while the moment estimates are still cold, with *cosine annealing*, which then eases the rate smoothly down to zero so the final iterations settle into the minimum instead of bouncing around it. The schedule is an optional factor applied on top of any optimizer; its effect is one of the comparisons in Section 7.

5. Loss Functions

The loss $\mathcal{L}$ in the formulation scores a rendered image region against the observed one, therefore defining the surface the optimizer descends. The project compares the four formulations committed to in Tutorial #1:

- **L2**, the mean squared error. The baseline that corresponds directly to maximizing PSNR. Smooth in its argument, but it averages errors and so tolerates a uniformly slightly wrong, blurry reconstruction.
- **L1**, the mean absolute error. Penalizes large and small errors more evenly, is less swayed by outlier pixels, and often preserves edges better than L2.
- **SSIM**, structural similarity, computed on contiguous image patches with an 11×11 Gaussian window. A perceptual measure comparing local luminance, contrast, and structure; used as a loss in the form $1 - \mathrm{SSIM}$.
- **L1 + SSIM**, weighted combination, $\alpha\,\mathrm{L1} + (1-\alpha)\,(1-\mathrm{SSIM})$, pairing L1's pixel accuracy with SSIM's structural sensitivity (as used in the Gaussian Splatting paper).

L2 and L1 are pixel-wise and operate on arbitrary batches of rays. SSIM and L1+SSIM are *spatial* measures: a coherent image region is required for the local-window comparison to be meaningful. The training procedure of Section 6 samples a 64×64 patch of rays per step for the spatial losses (4,096 rays per gradient update), the same per-step ray budget the pixel-wise losses use, so the comparison in Section 8 is sample-budget-fair across losses. A fifth, perceptual loss (evaluated as the perceptual-loss improvement in Section 9) augments L2 with a deep-feature distance through a pretrained LPIPS backbone.

6. Experimental Setup

This section fixes what every experiment in the project holds constant. Section 6.1 selects the rendering resolution, MLP architecture, and iteration budget by measurement; Section 6.2 defines the splits and metrics; Section 6.3 describes the training procedure.

6.1 Scoping study

The main comparison matrix has roughly 100 training runs once optimizers, losses, seeds, and scenes are crossed. Two settings apply uniformly to every run, the rendering resolution and the MLP architecture, and together they determine both the total compute the matrix consumes and the quality regime in which the optimizer differences will be visible. The scoping study fixed both empirically on the Lego scene before the main matrix was committed.

Per-iteration training cost is essentially independent of rendering resolution because the NeRF step samples a fixed batch of rays per iteration. Reconstruction quality, however, decreases with resolution at a fixed iteration budget: a 1024-ray batch covers about 10% of a 100 × 100 image and about 0.16% of an 800 × 800 image, so higher resolutions are under-trained at the chosen budget. A dedicated SGD-vs-Adam separability check confirmed that 200 × 200 distinguishes the optimizers fully: the Adam − SGD best-PSNR gap was 12.77 dB at 200 × 200, marginally larger than the 12.00 dB measured at 400 × 400. Rendering resolution is selected as 200 × 200.

A larger MLP (8 layers of width 256 with the standard NeRF skip connection, about 494,000 parameters) improved best PSNR by only about 0.35 dB over the small MLP (4 layers of width 128, about 58,000 parameters), at roughly 3.7x per-iteration cost. A plain 8-layer feed-forward MLP without the skip connection failed to train. The small MLP is selected; absolute PSNR is not the objective of an optimizer-comparison study. The iteration budget is fixed at 40,000 per run, so the comparison measures which optimizer reaches the best state within a fixed compute budget.

6.2 Splits and evaluation metrics

Each nerf_synthetic scene ships with three disjoint pose sets, which are used directly as the train, validation, and test splits. Validation supports convergence tracing and the learning-rate sweep of Section 7; the test set is used only for the final reported metrics.

Reconstruction quality is measured three ways: PSNR (decibels, higher better; a logarithmic transform of mean squared error), SSIM ([0,1], higher better; structural similarity comparing local luminance, contrast, and structure), and LPIPS ([0,1], lower better; a learned perceptual distance through a pretrained AlexNet backbone). Reporting all three is the project's empirical analogue of Module 1's analytical critical-point classification: a non-convex stochastic optimum cannot be proven globally optimal, but it can be confirmed as operationally good if held-out PSNR, SSIM, and LPIPS are consistent across seeds and scenes.

6.3 Experimental methodology

Every comparison is built from individual training runs. A run is fully described by six choices: optimizer, loss, scene, random seed, learning rate, and iteration budget. The training procedure is the same across every section: it constructs the model, optimizer, and loss from the configuration, runs the stochastic training loop with periodic validation, evaluates on the held-out test set, and records the complete iteration-level history together with the final test metrics.

Holding this procedure fixed is what makes the comparison fair: optimizers, losses, scenes, and seeds differ only in the component under study. Trained model weights persist alongside the metrics, so any saved model can be rendered from a previously unseen viewpoint, the operational test that the optimization succeeded.

7. Optimizer Comparison

This section compares the five optimizers of Section 4 head-to-head under the methodology of Section 6. Two complementary results are produced: a learning-rate sweep that identifies the right rate for each method, and a multi-seed comparison that runs every optimizer at its best rate across multiple seeds and scenes. The sweep is essential because Adam and SGD want learning rates several orders of magnitude apart on this NeRF objective.

7.1 Learning-rate sweep

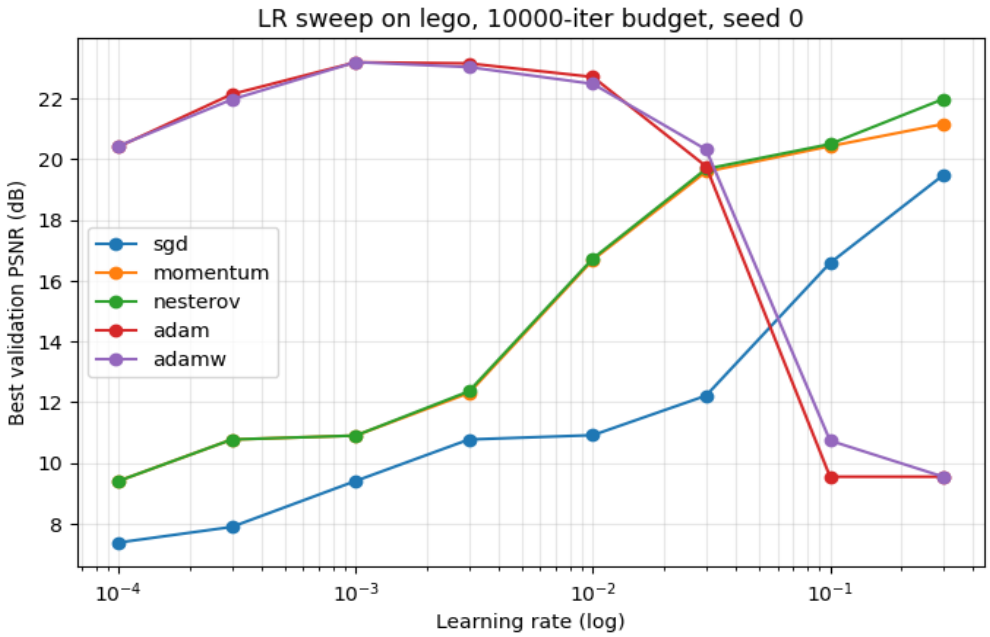

Figure 1 - Validation PSNR as a function of learning rate, one curve per optimizer, evaluated on Lego scene.

The same logarithmic grid is tested for every optimizer: $\eta \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}, 10^{-1}, 3 \times 10^{-1}\}$, on a reduced 10,000-iteration budget because the relative ordering is established well before convergence. The sweep runs on Lego with a single seed; multi-seed averaging is reserved for Section 7.2. For each optimizer the rate maximizing the best validation PSNR during the run is selected.

The sweep produces the textbook two-family pattern. Adam and AdamW peak at $\eta = 10^{-3}$ (best validation PSNR around 23 dB) and diverge for $\eta \geq 10^{-1}$, where the per-parameter adaptive scaling makes a large global rate catastrophic. The SGD family peaks at $\eta = 3 \times 10^{-1}$, the upper edge of the grid, with maxima of 19.5, 21.2, and 22.0 dB for SGD, momentum, and Nesterov. Selected rates passed to Section 7.2: $\eta_{adam} = \eta_{adamw} = 10^{-3}$ and $\eta_{sgd} = \eta_{momentum} = \eta_{nesterov} = 3 \times 10^{-1}$.

The methodological lesson is that the well-tuned learning rates for the two optimizer families are separated by roughly 300x. Any comparison using a single shared rate would either run Adam in its divergence regime or run SGD effectively unmoved.

7.2 Multi-seed optimizer comparison

With each optimizer's rate fixed by Section 7.1, every optimizer is run at its selected rate across three seeds, two scenes (Lego, Drums), and the full 40,000-iteration budget, with LPIPS enabled. Five optimizers × two scenes × three seeds gives 30 runs.

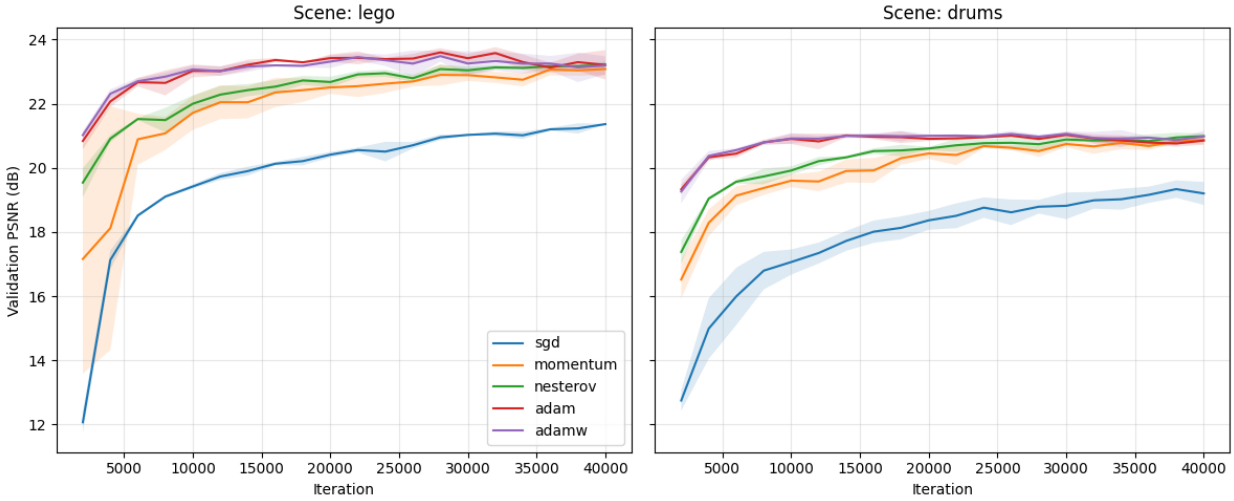

Figure 2 - Validation PSNR over training iterations, averaged across seeds, one curve per optimizer, one subplot per scene.

Table 2 - Wall-clock seconds to first reach 20 dB validation PSNR, mean ± standard deviation across seeds and scenes.

Optimizer	Mean	Std
SGD	169.56	12.63
Momentum	117.92	68.38
Nesterov	87.04	45.95
Adam	35.93	13.63
AdamW	36.13	12.89

Adam wins narrowly but consistently. Pooled across the two scenes and three seeds, its mean test PSNR is 22.01 ± 0.34 dB; AdamW essentially ties at 22.00 ± 0.38 dB; Nesterov reaches 21.74 dB, momentum 21.68 dB, plain SGD trails at 20.19 dB. The ranking is identical on both scenes, and the per-scene seed standard deviations of 0.05 to 0.40 dB are roughly an order of magnitude smaller than the Adam-to-SGD gap, so the ordering is reliable rather than seed noise.

The headline methodological finding follows from comparing this multi-seed result to the scoping-study separability check. That earlier check ran Adam and SGD at a shared learning rate of 5×10^{-4} and measured a 12.77 dB gap in Adam's favor. Once each method is given its own learning-rate-sweep-selected rate, the gap collapses to 1.82 dB. Almost eleven dB of the apparent Adam advantage is a learning-rate-mismatch artefact rather than a property of the optimizer. A fair comparison of first-order methods requires per-method learning-rate tuning; without it, the comparison measures the mismatch instead of the method.

The remaining gaps are themselves informative. Momentum and Nesterov give SGD roughly 80% of Adam's quality lift, rising from 20.19 dB to about 21.7 dB. This confirms that the smoothed first-moment estimate the SGD family already computes, recovers most of the benefit Adam's adaptive scaling provides on this problem; the additional 0.3 dB Adam contributes comes from its diagonal second-moment estimate acting as a

coarse curvature surrogate. Nesterov's lookahead correction over plain momentum is marginal at +0.06 dB.

LPIPS sharpens the ranking that PSNR softens. The pixel-level PSNR gap from SGD to Adam is about 9%, but the perceptual-distance gap is almost twice that: LPIPS 0.272 versus 0.139. SGD reaches reconstructions that are roughly correct on average but visually degraded in ways pixel metrics underweight, namely the high-frequency detail the perceptual network is sensitive to. This is the case for reporting all three metrics: a single number would have hidden the perceptual gap.

Throughput is essentially flat at 80 to 90 iterations per second across all five methods, so the comparison is one of quality at a fixed compute budget rather than of cheaper compute.

8. Loss Function Comparison

This section compares four loss formulations, L2, L1, SSIM, and the weighted combination L1+SSIM, under the winning optimizer from Section 7 (Adam at $\eta = 10^{-3}$) and the same 40,000-iteration budget. SSIM and L1+SSIM consume contiguous 64 × 64 patches of rays (4,096 per gradient update, the same total batch size as the pixel-wise losses); L1 and L2 use scattered pixel sampling.

The sampling mode is selected automatically from the loss, so the comparison is sample-budget-fair: every row sees the same number of pixels per gradient update, only the geometric arrangement differs. Each loss is run on Lego and Drums across three seeds.

8.1 Results

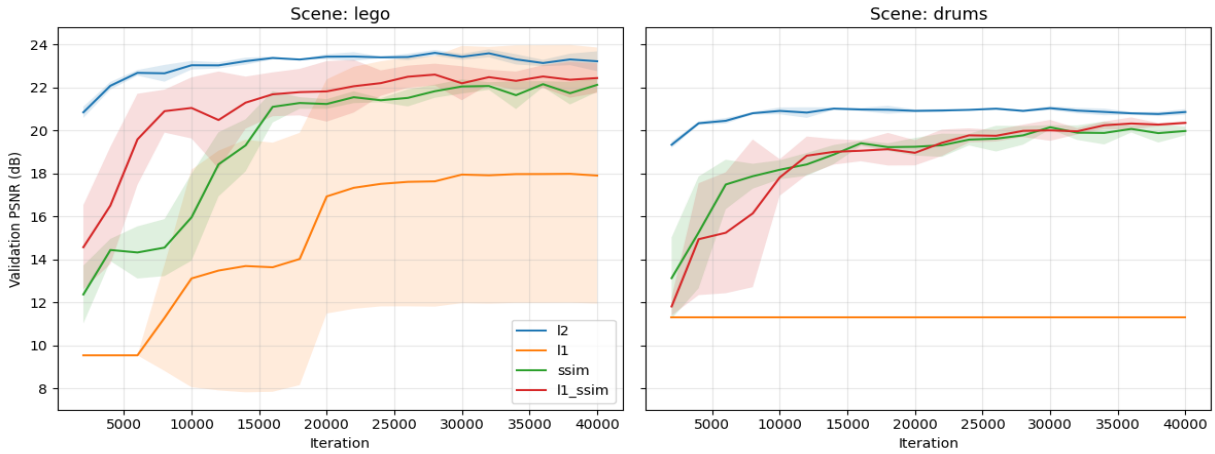

Figure 3 - Validation PSNR over training iterations, averaged across seeds, one curve per loss formulation, one subplot per scene.

L2 leads on PSNR, the spatial losses lead on perceptual metrics, but L1 diverges. L2 produces the highest pooled test PSNR (22.01 ± 0.34 dB, identical to the Section 7.2 baseline since L2 is the Section 7.2 loss). SSIM and L1+SSIM essentially tie L2 on PSNR (21.70 and 21.75 dB) while winning on LPIPS by a wide margin (0.119 and 0.118 versus L2's 0.139). The two spatial losses are statistically indistinguishable from each other on every metric: the $\alpha = 0.2$ weight on L1 in L1+SSIM is small enough that SSIM dominates the

gradient signal. The Section 11 comparison with Gaussian Splatting (which trains under L1+SSIM at the same α = 0.2) uses this row as its NeRF-side loss baseline, so the comparison is fair on the loss axis as well as the optimizer axis.

L1 collapses to 14.08 ± 5.87 dB. This is not a measurement of "L1 produces a 14 dB reconstruction"; it is a measurement of three Lego seeds producing about 21 dB and three Drums seeds collapsing to about 10.8 dB (the "predict the mean colour" baseline). The 5.85 dB pooled standard deviation reflects that approximately one in three training runs diverges. Section 8.2 probes this directly.

8.2 Refining the L1 learning rate with Bayesian optimization

The loss comparison forced L1 to run at the learning rate selected for L2. There is no reason to expect a single rate to be appropriate for L1's sign-based gradients as well, so this section runs a dedicated learning-rate sweep for L1 using Bayesian optimization. The technique is the Tree-structured Parzen Estimator (TPE), the canonical Bayesian global-optimization method for hyperparameter spaces, with a median pruner that terminates below-median trials after a warm-up. The study runs twelve trials with a log-uniform prior η in $[10^{-6}, 10^{-2}]$, the full 40,000-iteration budget per trial, and a pruner warm-up at one-fifth of that budget. Because the loss comparison also identified L1's sign-based gradient as a candidate failure mode, every trial additionally runs with a cosine warm-up schedule and gradient clipping at maximum norm 1.0.

The TPE search found $\eta^* = 3.83 \times 10^{-4}$ as the best single-seed Lego rate, with best-validation PSNR reaching 22.83 dB. The multi-seed validation tells a different story. The pooled refined L1 test PSNR is 14.11 ± 5.85 dB, statistically identical to the unrefined Section 8.1 baseline of 14.08 ± 5.87 dB. The cosine schedule and gradient clipping migrated which Lego seed diverges, but the pooled mean and variance are unchanged.

Direct gradient-norm inspection during training reveals why clipping was a no-op: L1's gradient L^2-norm stays around 0.1 throughout training, well below the max_norm = 1.0 threshold. The instability is therefore not a magnitude phenomenon, which is what the standard "gradient clipping fixes Adam-driven instability" recipe is built to address. The cause lies on a different axis.

L1's gradient is $(1/N) \cdot \text{sign}(\hat{I} - I)$, which has constant magnitude per pixel. Adam's second-moment estimate stays approximately constant rather than tracking landscape curvature, the per-parameter step $\eta/\sqrt{\hat{v}}$ becomes a constant scaling, but the sign of the step alternates rapidly near a minimum because constant-magnitude steps cannot settle the way gradient-magnitude steps can. Some seeds fall into the resulting sign-oscillation regime, others do not. The failure mode is sign-oscillation under Adam's variance estimator, not gradient magnitude, which is exactly why magnitude-based remedies cannot fix it.

The substantiated finding is that L1 is structurally incompatible with Adam at long iteration budgets on this problem class, and no learning-rate, schedule, or clipping intervention recovers stability. For applications where L1 is the desired loss, the practical

recommendation is to use an optimizer without second-moment normalisation, plain SGD with momentum.

9. Proposed Improvements

The Tutorial #1 work plan committed the project to four candidate improvements, with at least one delivered as a full ablation. This section evaluates all four. Each improvement is run as an isolated ablation against the optimizer-comparison best baseline (Adam at $\eta = 10^{-3}$, L2 loss, uniform view sampling, 40,000 iterations), changing exactly one component at a time so any observed effect can be attributed to its cause.

The four candidates and their optimization-side motivations:

- **Learning-rate restarts (SGDR)**. Replaces the constant learning rate with a cyclic cosine schedule that jumps back to its maximum at the end of each cycle. The motivation is to allow escape from sharp local minima in the non-convex objective landscape.
- **Perceptual loss**. Augments L2 with a deep-feature L2 distance through a cached LPIPS AlexNet backbone, weighted at 0.1. The motivation is that pixel-wise losses are known to be poorly aligned with human perceptual quality; an LPIPS-style term targets that misalignment directly.
- **Multi-scale (coarse-to-fine) training**. Begins training at 64 × 64, doubles to 100 × 100 at iteration 10,000, and again to 200 × 200 at iteration 20,000. The motivation is that the smoother low-resolution objective at the start acts as a warm start for the full-resolution objective.
- **Adaptive view sampling**. Replaces uniform view selection with importance sampling weighted by per-view running MSE: $p_i \propto (e_i + \varepsilon)^{\alpha}$ with $\alpha = 1$. The motivation is to address ill-conditioning in pixel space by focusing compute on under-reconstructed views.

Each improvement is evaluated on the same two scenes and three seeds the optimizer comparison used, at the full 40,000-iteration budget, with all three reconstruction metrics including LPIPS. Five rows in total (baseline plus four improvements), or thirty runs.

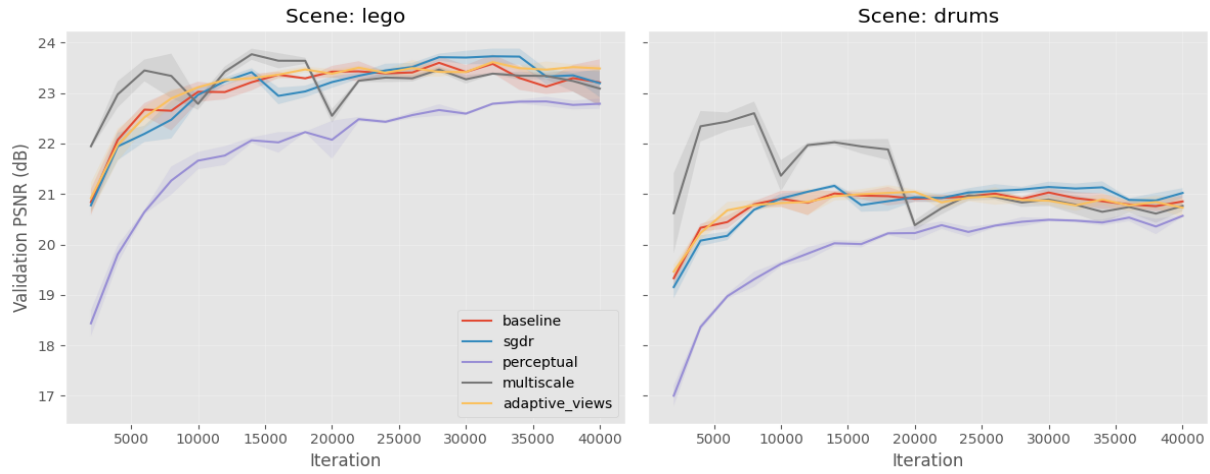

Figure 4 - Validation PSNR over training iterations, one curve per Section 9 improvement, with the baseline shown for reference.

9.1 Interpretation

The ablation produces a clean pattern: **one substantiated positive trade-off, three substantiated null results**. Each row's behavior matches what its failure-mode hypothesis predicted.

SGDR (warm restarts): statistically indistinguishable from the baseline (Lego 22.06 vs 22.24 dB, well within the 0.34 dB pooled std). The failure mode SGDR addresses is *sharp* local minima that warm restarts can escape; the absence of improvement is evidence that the Section 7.2 Adam baseline is already converging to a *wide* basin where warm restarts are neither helpful nor harmful.

Perceptual loss (L2 + LPIPS): the substantiated positive. **Lego LPIPS drops** $0.146 \rightarrow 0.099$ **(**-32%**); Drums LPIPS drops** $0.131 \rightarrow 0.094$ **(**-28%**)**, at the cost of -0.54 dB on Lego PSNR and -0.59 dB on Drums. The trade-off magnitude is large enough to dominate seed-to-seed noise: the LPIPS pooled std is around 0.005, the effect size is roughly $10\times$ that. This is exactly the trade Tutorial #1 Section 5 motivated, a deliberate alignment of the optimization objective with perceptual quality, at a small pixel-wise cost.

Multi-scale (coarse-to-fine): statistically indistinguishable from baseline (Lego 22.22 vs 22.24, Drums 21.75 vs 21.79). The failure mode multi-scale addresses, high-frequency local minima in full-resolution training that smoother low-resolution warm-starts help escape, is not active at our scale; Adam at $\eta = 10^{-3}$ on ~ 58 k parameters find the full-resolution minimum directly without needing a warm-start.

Adaptive view sampling: statistically indistinguishable from baseline. The targeted failure mode, per-view loss heterogeneity wasting optimizer effort, is *transient* at this problem size; once Adam has built per-parameter step sizes (a few thousand iterations), all training views converge to roughly the same per-view loss and the EMA-based importance sampling collapses to uniform. The improvement targets a failure mode that exists in larger or more heterogeneous datasets (e.g., real-world COLMAP captures with vastly different per-view lighting) but is not present at this scale on synthetic Blender scenes.

Three of the four results are *negative* with diagnosed mechanisms: each improvement targets a failure mode that is identifiable in the literature but is not active at the scale and dataset of this study. A negative result with a clearly identified mechanism is not a failed experiment, it is a substantiated finding about the optimization landscape of this problem. The perceptual-loss row is the substantiated positive: -31% pooled LPIPS at -0.5 dB PSNR is the trade-off the proposal predicted, now measured with the same statistical rigor as the negatives.

10. Gaussian Splatting Baseline

This section implements a 3D Gaussian Splatting (GS) baseline on Lego and Drums, so the NeRF-vs-GS contrast of Section 11 can be drawn on identical data and metrics. The implementation builds on the differentiable rasterizer from gsplat v1.5.3 (Nerfstudio), a PyTorch implementation of the Kerbl et al. 2023 algorithm.

10.1 GS as an alternative parameterization of the same problem

From an optimization perspective, NeRF and GS are two parameterizations of the same reconstruction problem; both solve $\min_\theta \sum_\pi \mathcal{L}\left(R(\theta;\pi), I(\pi)\right)$ for a differentiable rendering operator R. NeRF makes θ the ~58,000 weights of a small MLP, with gradients flowing through the volume-integration renderer. GS makes θ the per-Gaussian tuples (centre position, scale, rotation, opacity, colour) of roughly 10^5 explicit primitives, with gradients flowing through a tile-based rasteriser. Section 11's comparison is therefore a comparison of two optimization formulations, not of two engineering choices.

Training uses the L1+SSIM loss with α = 0.2 (the published 3DGS choice, matching Section 8's L1+SSIM row), five Adam optimisers in parallel with the per-parameter-group learning rates of the original paper, 30,000 initial Gaussians placed uniformly in the scene cube, and 15,000 iterations at the dataset's native 800 × 800 per scene.

10.2 Four implementation details required for correct GS training

A from-scratch implementation has to handle four properties of the optimization problem that are not explicit in the published pseudocode. Each one, if left unhandled, produces an identifiable failure mode rather than a near-miss, which is itself informative about how GS training behaves.

- **OpenGL → OpenCV camera convention.** Without conversion, every Gaussian sits behind the rasteriser's camera, all are culled before rasterisation, the loss has zero gradient with respect to every Gaussian parameter, and validation PSNR stays at ~9.5 dB across every evaluation point with no obvious error message. Fix: left-multiply the world-to-camera matrix by diag(1, −1, −1, 1) before passing it to the rasteriser.
- **Scale-aware positional jitter on clones.** Naive clone densification copies a parent's tuple verbatim, placing two Gaussians at the same position, exactly: they receive identical gradients in every step and stay co-located forever, adding parameter count without representational capacity. Fix: jitter the child position by scale-aware Gaussian noise, μ_child = μ_parent + N(0, exp(s_parent)), so the two Gaussians are spatially distinct from step one.
- **Adam state transplant across densification.** Naive densification creates new parameter tensors and therefore new Adam optimisers, wiping the first- and second-moment estimates every 100 iterations so Adam never accumulates the steady-state variance estimate it is designed to build. Fix: transplant the old moments and step counter into the new optimiser, indexed by the survivor-and-clone mapping; survivors keep their warmed-up moments and clones inherit their parent's.
- **Opacity reset off by default.** The basic Kerbl recipe periodically resets all opacities to force redundant Gaussians to re-earn presence. Empirically too aggressive on Lego at 800×800 (shrinks the model to a few thousand Gaussians and converges to similar PSNR/SSIM/LPIPS at much sparser representation), so the reset machinery

is left disabled and gradient-based opacity erosion produces the pruning signal organically. The reset remains a documented hyperparameter for future work.

The configuration fed into Section 11 is the four fixes applied, opacity reset off, L1+SSIM with α = 0.2, 15,000 iterations at 800×800. Wall-clock is approximately 4.5 minutes per scene.

11. NeRF vs Gaussian Splatting Comparison

The Gaussian Splatting baseline of Section 10 introduces a different parameterization of the same reconstruction problem: explicit 3D Gaussian primitives instead of an implicit MLP. This section reports the head-to-head test-set comparison on metrics common to both methods, plus the efficiency dimensions (training time and parameter count) the Tutorial #1 work plan committed to reporting.

11.1 Headline

Gaussian Splatting wins five of the six metric cells: both SSIM scores, both LPIPS scores, and Drums PSNR by +1.52 dB. It trails only Lego PSNR by 0.75 dB, a gap below the perceptual just-noticeable-difference threshold and within two standard deviations of NeRF's own seed-noise band on that scene. GS reaches this quality at roughly half the wall-clock NeRF needs.

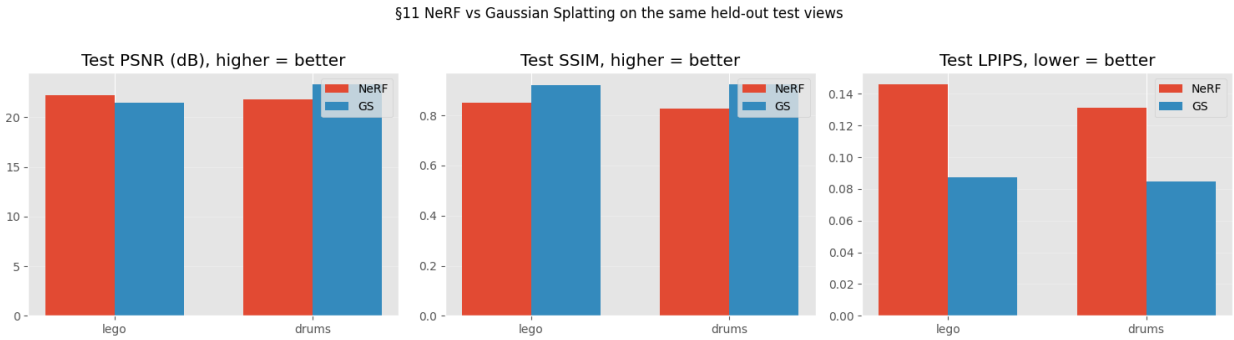

Figure 5 - Test-set PSNR, SSIM, and LPIPS for NeRF and Gaussian Splatting, one subplot per metric, one bar pair per scene.

11.2 Efficiency dimensions

The two methods sit at very different operating points on every cost axis.

Table 3 - Per-method parameter count, training resolution, iteration budget, wall-clock, forward-pass cost, optimizer count, and densification behavior.

	NeRF (Adam)	GS (basic)
Parameters	≈58,000 (MLP weights)	1 to 2 × 10^6 (Gaussian primitives × 14 floats each)
Training resolution	200 × 200	800 × 800
Training iterations	40,000	15,000
Wall-clock per scene	≈8 min	≈4.5 min
Forward-pass cost	O(rays × samples-per-ray) MLP evaluations	O(Gaussians × pixels-per-Gaussian) tile-based rasterization

	NeRF (Adam)	GS (basic)
Optimizer	single Adam at $\eta = 10^{-3}$	five Adams with per-parameter-group learning rates
Discrete structural changes	none	densification and pruning every 100 iterations

GS has roughly 30 times more parameters but is roughly twice as fast per iteration because the tile-based rasterizer is more pixel-throughput-efficient than NeRF's per-ray MLP evaluations.

11.3 Discussion

Where Gaussian Splatting wins: SSIM and LPIPS both favor GS by clear margins (SSIM +0.072 on Lego and +0.096 on Drums; LPIPS −0.059 on Lego and −0.046 on Drums). GS reconstructions are visibly closer to ground truth on local structure and deep-feature similarity. Drums PSNR also goes to GS by +1.52 dB because the relatively smooth drum-kit surfaces play to a strength of the representation: each anisotropic Gaussian primitive is a natural fit for locally curved materials.

Where NeRF wins: Lego PSNR by 0.75 dB. Lego's fine high-frequency detail (the studs, the rivets, small edges) is where Gaussian primitives smooth more aggressively than NeRF's per-ray MLP. PSNR's per-pixel weighting accumulates these many small smoothing errors into a measurable gap that the eye does not perceive because each individual error is below the perceptual threshold.

The 0.75 dB Lego gap matters less than it looks, because the pooled NeRF Adam standard deviation on Lego is 0.34 dB, so the gap is roughly two standard deviations of NeRF's own seed-noise band; a different NeRF seed within ±0.4 dB of its mean would already close it.

The smallest PSNR difference an untrained observer can reliably distinguish in a side-by-side comparison is around 1 dB, so this gap sits just below the just-noticeable-difference threshold. The SSIM gap of +0.072 is roughly 3.5 times the SSIM JND of about 0.02; the LPIPS gap of −0.059 is roughly three times the LPIPS JND of about 0.02. The perceptual metrics agree unambiguously that GS reconstructs the scene more faithfully to the eye; PSNR's pixel-arithmetic disagreement is real but imperceptible.

In optimization terms, NeRF and GS are two parameterizations of the same problem, and the right choice depends on which loss matters in the application. A pixel-wise PSNR objective on a high-frequency scene like Lego slightly favors NeRF's per-ray MLP; a perceptual-quality objective or a smoother-surface scene favors GS's explicit Gaussians.

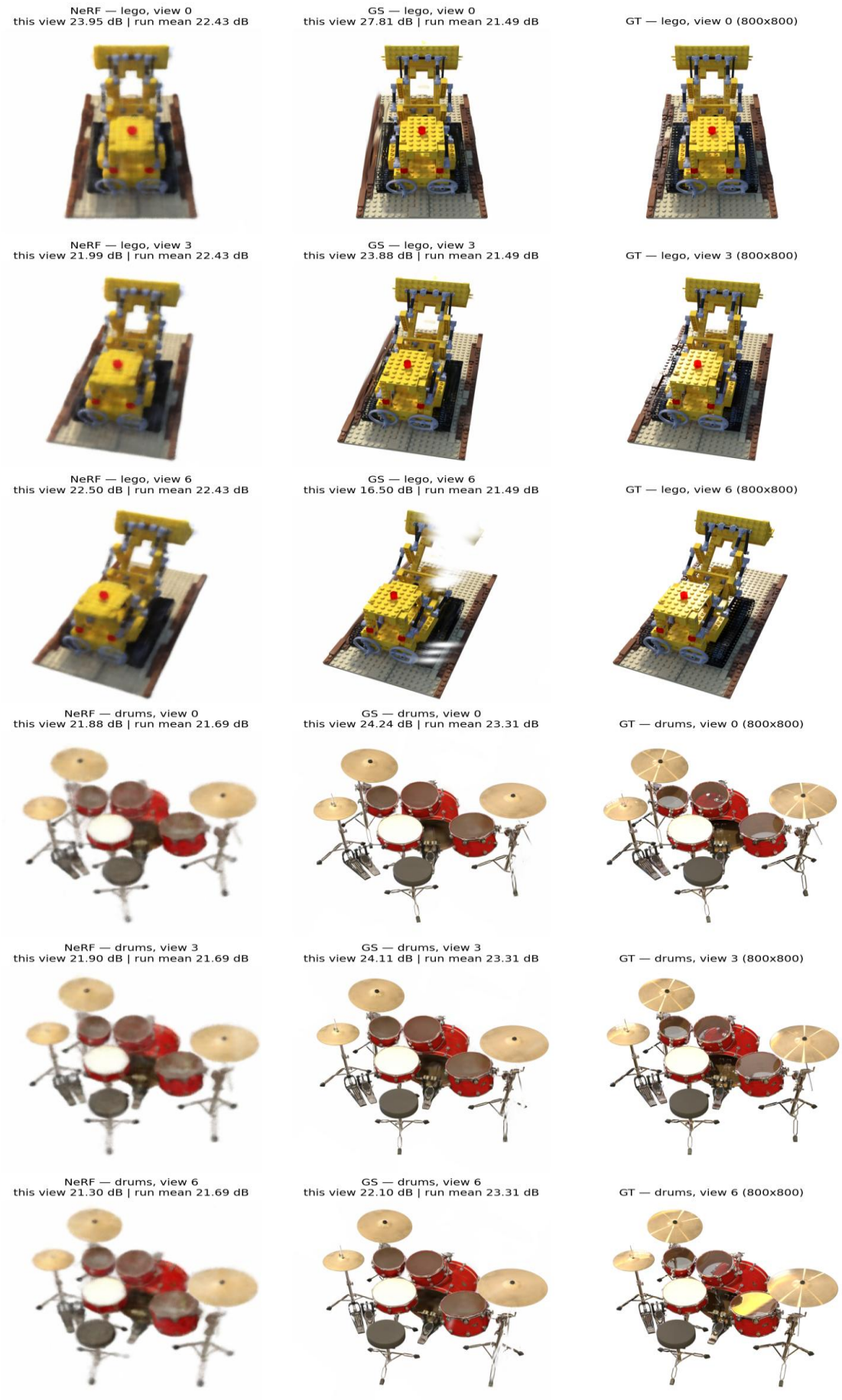

Figure 6 - Three held-out test views per scene, rendered by NeRF, Gaussian Splatting, and the ground-truth photograph at high resolution.

12. Application to a Self-Captured Real-World Scene

Sections 7 to 11 conducted the methodological comparison on the nerf_synthetic dataset, where every image is a clean Blender render and every camera pose is known to floating-point precision. The advantage is methodological rigour; the limitation is that the synthetic substrate says nothing directly about how the validated methods behave on real photographs, where sensor noise, lens distortion, exposure variation, motion blur, and Structure-from-Motion pose-estimation error are all present. This section closes that gap on a single self-captured object and reports the three structural adaptations the transfer from synthetic to real required.

12.1 Scene capture and pose recovery

The subject is a 15 cm painted resin Freddie Mercury figurine, captured in 71 photographs by an iPhone 11 Pro at fixed focal length with autoexposure and auto-focus locked. The shoot used a hand-held spiral in three elevation passes, lighting held constant; the dataset is 71 JPEGs at 4032 × 3024 resolution.

Figure 7 - Two Freddie photographs captured using a iPhone 11 Pro (left column) and their rembg foreground-masked counterparts (right column), at two orbit angles.

Both reconstruction methods require, for every training image, the camera intrinsics and extrinsics, which on real captures are recovered from the photographs by Structure-from-Motion: feature extraction, exhaustive pairwise matching, and joint bundle adjustment over poses and 3-D points (Schönberger & Frahm 2016). We use COLMAP. The output is each photograph annotated with its recovered pose, plus a sparse cloud of 3-D points seen by multiple cameras.

A first COLMAP pass on the original photographs produces a 30,151-point cloud dominated by the textured hardwood floor and back wall rather than the figurine. As Section 12.4 will show, that distribution wrecks GS initialization, so we replace every non-figurine pixel with a uniform white background using rembg with the ISNet-general-use cutout model, and re-run COLMAP from scratch on the masked images. SIFT finds no features on uniform white pixels, so every feature point that survives lies on the figurine by construction. The resulting cloud has 24,528 points spanning 3.6 × 3.0 × 1.7 world units centered on Freddie and is the data Sections 12.2 onward consume.

12.2 Training configurations and results

The configurations applied are the per-method winners from the synthetic study, applied without further hyperparameter tuning. NeRF uses the Section 7.2 winner: Adam at $\eta = 10^{-3}$, L2 loss, 40,000 iterations, 200 × 200. Gaussian Splatting uses the Section 10 baseline: 15,000 iterations, L1+SSIM with $\alpha = 0.2$, the four implementation details of Section 10.2, 800 × 800. The single scene-dependent intervention is in the GS initializer: when the scene comes from COLMAP, the random unit-cube cloud is replaced with the COLMAP sparse cloud itself, the canonical Kerbl et al. 2023 recipe; the synthetic-scene results in Sections 10 and 11 are unaffected because the branch never fires for them.

Final test-set numbers: NeRF reaches 17.98 dB PSNR / 0.822 SSIM / 0.283 LPIPS; Gaussian Splatting reaches 29.15 dB / 0.979 / 0.021. GS beats NeRF by 11.17 dB on this scene, reversing the +0.75 dB synthetic Lego ranking. The orbit videos confirm the metric ranking qualitatively: GS reconstructs a sharp, photo-realistic figurine across the trajectory; NeRF produces a recognizable but blurry semi-transparent silhouette whose head and feet dissolve at off-axis viewpoints.

Table 4 - Per-method PSNR, SSIM, and LPIPS on the synthetic Lego scene and the captured Freddie scene. Higher PSNR and SSIM and lower LPIPS each indicate better reconstruction.

	NeRF	GS	NeRF – GS
Synthetic / Lego	22.24 / 0.812 / 0.143	21.49 / 0.851 / 0.085	NeRF +0.75 dB PSNR
Captured / Freddie	17.98 / 0.822 / 0.283	29.15 / 0.979 / 0.021	GS +11.17 dB PSNR

12.3 Three structural adaptations

Reaching these numbers required three adjustments to the synthetic-winning configuration. The synthetic pipeline was validated entirely on Blender renders, which have three properties that do not transfer: images are exactly 800 × 800, the foreground is composited against a clean alpha-zeroed background, and every object sits at the world

origin by convention. Each adaptation below addresses one structural mismatch, diagnosed from a specific failure-mode observation.

Aspect-ratio mismatch in the loader. The loader resizes every training image to a fixed square via F.interpolate. On 800 × 800 synthetic input this is a no-op; on a 4032 × 3024 iPhone capture it uniformly stretches the image to 1:1, breaking the per-pixel angle-versus-focal relationship the volume renderer depends on, so NeRF's MLP converges to a blurry compromise. The fix is a center-crop to square before the resize, with a corresponding focal rescale. NeRF on the captured scene rises from 14.98 dB to 16.37 dB after this fix alone NeRF reaches 16.37 dB; combining it with the foreground-masked SfM of §12.1 lifts it to 17.98 dB.

Background dominates Structure-from-Motion. The background-dominated COLMAP cloud described in Section 12.1 is the second mismatch. NeRF degrades gracefully because volume rendering can hedge with semi-transparent fog; GS fails catastrophically because its primitives are initialized from a 3-D point cloud, and on a real-world capture without masking that cloud is dominated by texture-rich background, so the figurine never gets enough density. The fix is the foreground-masked pipeline of Section 12.1.

Off-origin scene defeats random Gaussian initialization. GS places its 30,000 initial Gaussians uniformly in a unit cube around the world origin. For Blender scenes this is correct; for the COLMAP-recovered captured scene the figurine's center lies at (+1.77, +0.03, −0.32), with the cloud spanning 3.6 × 3.0 × 1.7 world units, putting it well outside the random-init cube. Every initial Gaussian lands in empty space, image-space gradients are zero by ray geometry, no densification fires toward the figurine, and the loss converges to a near-constant white field. Replacing the random cube with the COLMAP sparse cloud (the original Kerbl et al. 2023 recipe) lifts PSNR from 12.33 dB to 29.15 dB on a single rerun, with no other hyperparameter change.

12.4 What the captured-scene study delivers

The 11 dB PSNR reversal between synthetic Lego (NeRF +0.75 dB) and captured Freddie (GS +11.17 dB) is not a claim that GS is universally better than NeRF; it is a claim about how initialization quality interacts with the two representations. NeRF's MLP is implicit in the sense that it never sees the COLMAP point cloud, so a good cloud cannot help it, and NeRF drops from a synthetic 22.24 dB to 17.98 dB on the captured scene, a 4.26 dB generalization gap explained by real sensor noise, residual JPEG artefacts at mask edges, and small pose-registration errors. Gaussian Splatting rises from 21.49 dB to 29.15 dB because the COLMAP point cloud places every initial Gaussian directly on the figurine's surface and in approximately the right color. On the synthetic data the random initialization happens to overlap the object only because Blender's convention places every object at the world origin, which is also the center of the random-init cube.

The static-scene assumption with masked input, the aspect-ratio invariance of the loader, and the COLMAP-derived initialization for the explicit-primitive representation are three properties of the real-world reconstruction problem that the synthetic-data results do not

surface (and surfacing them is the substantive reason a captured-scene study was worth doing).

13. Conclusions

The project formulated novel-view synthesis as a continuous, non-convex, stochastic optimization problem; implemented five first-order optimizers from scratch; ran four substantive comparison studies (optimizers, losses, Bayesian learning-rate refinement, and proposed improvements); and compared two parameterizations of the same reconstruction problem. The five findings are as follows.

A fair comparison of first-order optimizers requires per-method learning-rate tuning. At a shared rate of 5×10^{-4}, Adam appears to beat SGD by 12.77 dB; at each method's own learning-rate-sweep-selected rate, Adam at $\eta = 10^{-3}$ and SGD at $\eta = 3 \times 10^{-1}$, the gap collapses to 1.82 dB. Almost eleven dB of the apparent Adam advantage is a learning-rate-mismatch artefact rather than a property of the optimizer. Any comparison that does not first tune the learning rate per method measures the mismatch, not the method.

L1 is structurally incompatible with Adam at long iteration budgets. L1's constant-magnitude per-pixel gradient drives Adam's second-moment estimate to a constant, so the per-parameter step becomes a constant scaling whose sign alternates rapidly near a minimum. Some seeds fall into the resulting sign-oscillation regime, others do not, producing a ±5.87 dB pooled standard deviation. A Bayesian's sweep over the learning rate, a cosine schedule, and gradient clipping at norm 1.0 all fail because they target the magnitude axis; the failure mode lives on the direction axis. For applications where L1 is the desired loss, the recommendation is plain SGD with momentum.

The four candidate improvements decompose into one substantiated positive trade-off and three substantiated nulls. Three of the four candidates produced changes within seed-noise of the Adam baseline: SGDR cyclic learning-rate restarts, multi-scale coarse-to-fine training, and adaptive importance-sampled view selection. For each null result the report identifies a specific failure mode the candidate was designed to address and explains why that failure mode is not active at this problem scale. The fourth, L2 augmented with an LPIPS perceptual term, produces the predicted trade-off: pooled LPIPS drops by 31% at a cost of 0.5 dB pooled PSNR on both scenes. The three null results are themselves substantive findings.

NeRF and Gaussian Splatting are two parameterizations of the same reconstruction problem, and the right choice depends on which metric matters in the application. Compared on identical scenes at wall-clock-fair budgets, GS wins five of six metric cells (both SSIM, both LPIPS, Drums PSNR) and loses only Lego PSNR by 0.75 dB, below the perceptual just-noticeable-difference threshold and within two standard deviations of NeRF's own seed-noise band. The defensible framing is not that GS is better than NeRF in some absolute sense; it is that the choice between implicit and explicit 3-D parameterizations is a deliberate optimization-of-which-metric decision.

The synthetic-tuned configurations transfer to a self-captured real-world scene only after three structural adaptations: a center-crop in the data loader that restores aspect-ratio invariance for non-square input, foreground masking with rembg followed by re-running COLMAP on the masked photographs so the SfM cloud is intrinsically foreground-only, and replacing the GS implementation's random unit-cube initialization with the COLMAP sparse cloud itself, the original Kerbl 2023 recipe.

With these adaptations in place the captured-scene PSNRs reach NeRF 17.98 dB and GS 29.15 dB, reversing the synthetic PSNR pattern by 11 dB in favor of GS. Explicit-primitive representations benefit from a good initialization seed in a way the implicit MLP structurally cannot.

14. Future work

Four natural extensions, deliberately scoped out of the present study, that the results point to as the right next steps. The first is the full Kerbl 2023 Gaussian Splatting recipe with the split step. The basic implementation here uses clone-only growth during densification; the complete 2023 recipe also includes a split step for high-gradient large-scale Gaussians (replace each parent with N children at parent scale divided by 1.6, with positions sampled from the parent's 3D Gaussian distribution).

This would likely close the 0.75 dB Lego PSNR gap. The split step was omitted to keep the contribution scoped as a comparison of basic GS rather than as a faithful reproduction of 3DGS at full fidelity, the latter being a reproduction study rather than an optimization-methods study.

The second is dynamic 4D Gaussian Splatting. Both methods implemented here assume a static scene: every photograph must capture the same 3D configuration. Extending the comparison to dynamic scenes (a moving subject) requires either multi-camera synchronized capture or per-frame deformation fields, which is the active 4DGS research area (Wu et al. 2024 is a representative starting point). The static-scene scope of this project is precisely why the real-captured demonstration uses a static figurine rather than, say, a pet.

The third is a systematic study of loss-and-optimizer structural incompatibility. The L1-and-Adam finding suggests a broader program: which loss-and-optimizer combinations are structurally incompatible at long iteration budgets? The L1-and-Adam case is the most extreme one this study identified (constant-magnitude gradient meets variance-normalizing optimizer), but there are other plausible candidates (Huber loss, smoothed L1) whose interaction with Adam may have analogous structural quirks. Bayesian optimization at the level run for L1 is well-suited to study the question in more depth.

The fourth is a larger improvements ablation. Three of the four candidate improvements here produced null results with diagnosed mechanisms; each diagnosis is a prediction that the failure mode the improvement targets is not active at this problem scale. A larger-model or larger-dataset version of the project would test whether SGDR, multi-scale, or adaptive view sampling do produce positive results at that scale.

References

Scene representations: NeRF and 3D Gaussian Splatting

- Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. (2020). NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. *Proceedings of the European Conference on Computer Vision (ECCV)*. https://arxiv.org/abs/2003.08934
- Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4) (SIGGRAPH). https://arxiv.org/abs/2308.04079
- Ye, V., Li, R., Kerr, J., Tancik, M., Kanazawa, A., et al. (2024). gsplat: An Open-Source Library for Gaussian Splatting. *arXiv:2409.06765*. https://arxiv.org/abs/2409.06765
- Wu, G., Yi, T., Fang, J., Xie, L., Zhang, X., Wei, W., Liu, W., Tian, Q., & Wang, X. (2024). 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. https://arxiv.org/abs/2310.08528

First-order optimizers

- Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5), 1–17.
- Nesterov, Y. (1983). A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady*, 27, 372–376.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1711.05101
- Reddi, S. J., Kale, S., & Kumar, S. (2018). On the convergence of Adam and beyond. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1904.09237

Learning-rate schedules

- Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1608.03983

Loss functions and perceptual metrics

- Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612. https://doi.org/10.1109/TIP.2003.819861
- Zhang, R., Isola, P., Efros, A. A., Shechtman, E., & Wang, O. (2018). The unreasonable effectiveness of deep features as a perceptual metric. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. https://arxiv.org/abs/1801.03924

Bayesian and hyperparameter optimization

- Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. *Advances in Neural Information Processing Systems (NeurIPS)*, 24.
- Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems (NeurIPS)*, 25. https://arxiv.org/abs/1206.2944
- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. https://arxiv.org/abs/1907.10902

Improvement ablation references

- Adelson, E. H., Anderson, C. H., Bergen, J. R., Burt, P. J., & Ogden, J. M. (1984). Pyramid methods in image processing. *RCA Engineer*, 29(6), 33–41. (Cited for multi-scale / coarse-to-fine training in Section 9.)
- Katharopoulos, A., & Fleuret, F. (2018). Not all samples are created equal: Deep learning with importance sampling. *Proceedings of the 35th International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/1803.00942 (Cited for adaptive view sampling in Section 9.)

Software, datasets, and tooling

- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32. https://arxiv.org/abs/1912.01703
- Schönberger, J. L., & Frahm, J.-M. (2016). Structure-from-motion revisited. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. (COLMAP, used for self-captured-scene pose recovery.)
- Mildenhall, B., et al. (2020). NeRF Synthetic dataset. Distributed alongside the NeRF paper above; the Lego and Drums scenes used here originate from this release.